



中国矿业大学  
CHINA UNIVERSITY OF MINING AND TECHNOLOGY

# 《大数据可视化》综合设计报告七

## 题目：高性能分布式计算实践

学 院 计算机科学与技术学院

班 级 2023 级大数据一班

姓 名 王志豪

学 号 08232337

中国矿业大学

2026 年 05 月

课程设计报告格式要求：

1. 报告中应包含一、二级标题；
2. 正文宋体小 4 号，1.5 倍行距；
3. 报告正文不少于 1000 字；
4. 上交报告电子版：<https://v.wjx.cn/vm/mv3f28U.aspx>，截止 4 月 26 日。

课程设计报告评分要点：

1. 报告中可视化技术的应用情况（20%）
2. 报告中可视化应用功能有效及完整性（40%）
3. 报告中的个人总结及体会情况（20%）
4. 报告格式规范程度（20%）

评语：

评分：

日期：

## 目录

|                  |    |
|------------------|----|
| 目录.....          | 2  |
| 1. 背景及实验概要 ..... | 3  |
| 1.1 背景.....      | 3  |
| 1.2 实验概要.....    | 3  |
| 2. 实验环境.....     | 4  |
| 3. 实验步骤.....     | 4  |
| 4. 总结及体会.....    | 12 |
| 5. 思考题.....      | 12 |

# 1. 背景及实验概要

## 1.1 背景

随着大数据、人工智能以及云计算技术的快速发展，海量数据处理需求不断增加。传统单机计算模式在处理大规模数据时面临计算能力不足、执行效率低等问题，因此分布式计算技术成为解决大规模数据处理问题的重要手段。MPI（Message Passing Interface，消息传递接口）是目前高性能计算领域应用最广泛的并行编程标准之一，通过消息传递机制实现多节点协同计算，提高计算资源利用率和程序运行效率。

WordCount（词频统计）是分布式计算领域的经典案例，其主要任务是统计文本中各单词出现的次数。通过将文本数据划分到多个计算节点并行处理，可以显著提升统计效率。因此，本实验基于 MPI 构建分布式计算环境，实现 WordCount 程序的并行化执行，掌握 MPI 集群环境搭建、节点通信以及分布式任务执行过程，为后续学习高性能计算与并行程序设计奠定基础。

## 1.2 实验概要

本实验基于 MPI 并行计算框架，采用一主两从的集群结构搭建分布式实验环境。首先利用 VMware Workstation 创建三台 Ubuntu Linux 虚拟机，分别作为主节点和两个从节点；随后配置节点间网络通信环境，实现 SSH 免密登录；接着安装 OpenMPI 运行环境，完成 MPI 集群配置；最后编译并运行 WordCount 程序，通过多个 MPI 进程协同完成文本词频统计任务。实验过程中重点完成以下内容：

- （1）搭建由 1 个主节点和 2 个从节点组成的 MPI 集群环境；
- （2）配置节点间 SSH 免密通信机制；
- （3）安装并配置 OpenMPI 运行环境；
- （4）编译并部署 WordCount 程序；
- （5）利用 MPI 实现多节点并行词频统计；
- （6）验证分布式环境下程序运行结果的正确性。

通过本实验，进一步理解 MPI 消息传递机制、分布式计算模式以及集群环境部署方法，掌握并行程序开发与运行的基本流程。

## 2. 实验环境

本实验采用 VMware Workstation 虚拟化平台搭建分布式计算环境，操作系统选用 Ubuntu Linux。实验环境由三台虚拟机构成，其中一台作为主节点，两台作为从节点，各节点通过 VMware NAT 网络模式进行通信。

硬件与软件环境配置如下：

表 2-1 实验环境配置

| 项目     | 配置                     |
|--------|------------------------|
| 虚拟化平台  | VMware Workstation     |
| 操作系统   | Ubuntu Linux 22.04 LTS |
| 集群结构   | 1 个主节点 + 2 个从节点        |
| 网络模式   | NAT                    |
| 主节点名称  | wangzhihao-1-master    |
| 从节点名称  | wangzhihao-2           |
| 从节点名称  | wangzhihao-3           |
| MPI 实现 | OpenMPI                |
| SSH 服务 | OpenSSH Server         |
| 编译器    | GNU GCC/G++            |
| 开发语言   | C++                    |
| 实验项目   | MPI 并行 WordCount 词频统计  |

各节点通过局域网互联，并配置 SSH 免密登录，实现主节点对从节点的远程管理与 MPI 任务调度。OpenMPI 运行环境部署于所有节点，实现多进程并行计算与消息通信。

## 3. 实验步骤

### (1). 准备主节点与从节点

首先准备一个主节点 WSL 和另外两个从节点 Ubuntu 虚拟机：

```
wang@wangzhihao-1-master:~$ hostname  
wangzhihao-1-master
```

```
wang@wang-VMware-Virtual-Platform:~$ hostname
wangzhihao-3

wang@wang-VMware-Virtual-Platform:~$ hostname
wangzhihao-2
```

## (2). 配置节点间 SSH 互通

- 生成 WSL 端密钥:

```
wang@wangzhihao-1-master:~$ ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/home/wang/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/wang/.ssh/id_rsa
Your public key has been saved in /home/wang/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:TYuK4tbfNJq0e3/bRadvYZDPH5lvKYAq3DVB9LHR9tQ wang@wangzhihao-1-master
The key's randomart image is:
+---[RSA 3072]---+
|      .. o.   .|
|      .. +o .E|
|      . .o. +  |
|      = . o .  |
|      S =  =+.|
|      . . + . .+B.|
|  ..o o = . . +.+|
|  ....+.B ... o o+|
|  .. o=*..o.... .o.|
+-----[SHA256]-----+
```

- 将公钥发到 wangzhihao-2 和 wangzhihao-3

从下面可以看到公钥已经成功发送，在 WSL 上可以无密码地 ssh 连接到两个主机。

```
wang@wangzhihao-1-master:~$ ssh-copy-id wang@192.168.244.133
The authenticity of host '192.168.244.133 (192.168.244.133)' can't be established.
ED25519 key fingerprint is SHA256:3H4CefCso3hEtVTmd5tAgNoh8Cr0PA6Dk38RkG/Dw4M.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter
out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompt
ed now it is to install the new keys
wang@192.168.244.133's password:

Number of key(s) added: 1

Now try logging into the machine, with:  "ssh 'wang@192.168.244.133'"
and check to make sure that only the key(s) you wanted were added.

wang@wangzhihao-1-master:~$ ssh-copy-id wang@192.168.244.134
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter
out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompt
ed now it is to install the new keys
wang@192.168.244.134's password:

Number of key(s) added: 1

Now try logging into the machine, with:  "ssh 'wang@192.168.244.134'"
and check to make sure that only the key(s) you wanted were added.
```

```
wang@wangzhihao-1-master:~$ ssh wang@192.168.244.133 hostname
wangzhihao-2
wang@wangzhihao-1-master:~$ ssh wang@192.168.244.134 hostname
wangzhihao-3
```

### (3). 安装 MPI

- 在三台主机上安装 MPI:

```
wang@wang-VMware-Virtual-Platform:~$ sudo apt install mpich -y
正在读取软件包列表... 完成
正在分析软件包的依赖关系树... 完成
正在读取状态信息... 完成
```

- 检查安装成功

```
wang@wang-VMware-Virtual-Platform:~$ mpirun --version
mpirun (Open MPI) 4.1.6

Report bugs to http://www.open-mpi.org/community/help/
```

```
wang@wang-VMware-Virtual-Platform:~$ mpic++ --version
g++ (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

#### (4). 创建源代码

- 创建目录

```
wang@wangzhihao-1-master:~$ ls wangzhihao-1-master/
wordcount
```

- 创建存放测试数据目录

```
wang@wangzhihao-1-master:~$ ls wangzhihao-1-master/wordcount/project_file/
big_file  small_file
```

- 创建 WordCount 源码

```
wang@wangzhihao-1-master:~$ head wangzhihao-1-master/wordcount/wordcount.cpp
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/time.h>

#include <string>
#include <cstring>
#include <fstream>
#include <sstream>
#include <iterator>
```

- 代码内容简述

##### 1. 辅助函数

| 函数         | 功能                                                  |
|------------|-----------------------------------------------------|
| getFiles() | 遍历文件夹，获取所有文件路径                                      |
| readFile() | 读取整个文件内容为字符串                                        |
| split()    | 按空格/换行分割字符串，返回单词数组                                  |
| getWords() | <b>负载分配：</b> 将单词列表按 rank 取模，分配给不同进程（但这里代码有个小问题，后面说） |



|               |                             |
|---------------|-----------------------------|
| countWords()  | <b>核心统计：</b> 用 map 统计单词出现次数 |
| mergeCounts() | 合并两个统计结果（汇总到同一个 map）        |

## 2. MPI 通信函数

| 函数                | 功能                                           |
|-------------------|----------------------------------------------|
| sendLocalCounts() | 将本进程的统计结果（单词 + 次数）发送给指定进程                    |
| recvCounts()      | 从指定进程接收统计结果                                  |
| treeMerge()       | <b>树形归约：</b> 采用类似二叉树的通信模式，将所有进程的结果汇总到 rank 0 |

目录结构：

```

├─ wordcount.cpp          (你贴的这个代码文件)
├─ project_file/
│   └─ big_file/
│       └─ big_100.txt    (大文件，放这里)
│           └─ small_file/
│               └─ file1.txt    (小文件，放这里)
│                   └─ file2.txt
│                       └─ ...    (任意多个小文件)

```

### (5). 创建测试数据

#### ● 创建大文件

```

wang@wangzhihao-1-master:~$ head wangzhihao-1-master/wordcount/project_file/big_file/big_100.txt
hello world hello mpi
hello world huawei
mpi mpi word count

```

#### ● 创建小文件

```

wang@wangzhihao-1-master:~/wangzhihao-1-master/wordcount/project_file/small_file
$ cat file1.txt file2.txt file3.txt file4.txt
hello world
hello mpi
world mpi mpi
word count mpi

```

### (6). 创建 makefile

```
wang@wangzhihao-1-master:~/wangzhihao-1-master/wordcount$ head Makefile
CC = mpic++

CCFLAGS = -O2

all: wordcount

wordcount: wordcount.cpp
        ${CC} ${CCFLAGS} wordcount.cpp -o wordcount

clean:
```

(7). 建立主机配置文件

```
localhost slots=2
192.168.244.133 slots=2
192.168.244.134 slots=2
```

(8). 下载 OpenBLAS

```
wang@wangzhihao-1-master:~/OpenBLAS-0.3.8$ ls /opt
OpenBLAS
```

软连接:

```
wang@wangzhihao-1-master:~/OpenBLAS-0.3.8$ sudo ln -s /opt/OpenBLAS/lib/libopenblas.so /usr/lib/libopenblas.so

wang@wangzhihao-1-master:~/OpenBLAS-0.3.8$ tail ~/.bashrc
if ! shopt -oq posix; then
    if [ -f /usr/share/bash-completion/bash_completion ]; then
        . /usr/share/bash-completion/bash_completion
    elif [ -f /etc/bash_completion ]; then
        . /etc/bash_completion
    fi
fi

export CPLUS_INCLUDE_PATH=$CPLUS_INCLUDE_PATH:/opt/OpenBLAS/include
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/opt/OpenBLAS/lib
```

(9). 进行编译

```
wang@wangzhihao-1-master:~/wangzhihao-1-master/wordcount$ make
mpic++ -O2 wordcount.cpp -o wordcount
```

(10). 将项目文件发给从节点

```
wang@LAPTOP-GGAVADQD:~/wangzhihao-1-master$ scp -r ./wordcount wang@192.168.244.134:~
wordcount                                100% 130KB 13.5MB/s 00:00
wordcount.cpp                            100% 8090 3.3MB/s 00:00
hostfile                                 100% 48 62.8KB/s 00:00
file3.txt                                100% 14 12.7KB/s 00:00
file1.txt                                100% 12 11.8KB/s 00:00
file4.txt                                100% 15 16.2KB/s 00:00
file2.txt                                100% 10 6.8KB/s 00:00
big_100.txt                              100% 60 54.8KB/s 00:00
Makefile                                  100% 139 115.2KB/s 00:00
wang@LAPTOP-GGAVADQD:~/wangzhihao-1-master$ scp -r ./wordcount wang@192.168.244.133:~
wordcount                                100% 130KB 22.9MB/s 00:00
wordcount.cpp                            100% 8090 8.9MB/s 00:00
hostfile                                 100% 48 70.0KB/s 00:00
file3.txt                                100% 14 21.3KB/s 00:00
file1.txt                                100% 12 18.9KB/s 00:00
file4.txt                                100% 15 15.1KB/s 00:00
file2.txt                                100% 10 14.8KB/s 00:00
big_100.txt                              100% 60 67.6KB/s 00:00
Makefile                                  100% 139 112.6KB/s 00:00
```

## (11). 运行检测

编写如下运行脚本:

```
app=${1}

if [ ${app} = "wordcount" ]; then
    mpirun --hostfile hostfile -np ${2} ./wordcount
fi
```

bash run.sh wordcount 2:

```
wang@wangzhihao-1-master:~/wangzhihao-1-master/wordcount$ bash run.sh wordcount
2
Authorization required, but no authorization protocol specified
Authorization required, but no authorization protocol specified
Authorization required, but no authorization protocol specified
Authorization required, but no authorization protocol specified
Authorization required, but no authorization protocol specified
Authorization required, but no authorization protocol specified
Authorization required, but no authorization protocol specified
Authorization required, but no authorization protocol specified
word count: 2.945 ms
word count: 0.86 ms
```

bash run.sh wordcount 4:

```
wang@wangzhihao-1-master:~/wangzhihao-1-master/wordcount$ bash run.sh wordcount 4
Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

word count: 21.157 ms
word count: 3.788 ms
```

bash run.sh wordcount 6:

```
wang@wangzhihao-1-master:~/wangzhihao-1-master/wordcount$ bash run.sh wordcount 6
Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

Authorization required, but no authorization protocol specified

word count: 44.752 ms
word count: 8.516 ms
```

通过上述运行，可以看出 wordcount 算法程序已经在集群中并行运行起来。

其中第一行输出代表的是大文件统计耗时，第二行输出代表的是小文件统计耗时。

## 4. 总结及体会

通过本次 MPI 分布式 WordCount 实验，我掌握了 MPI 集群环境的搭建方法，熟悉了 Linux 系统下 OpenMPI 的安装与配置流程，并成功实现了多节点之间的 SSH 免密登录和并行程序运行。在实验过程中，我深入了解了 MPI 消息传递机制以及主节点与从节点之间的协同工作方式，认识到分布式计算能够有效提高大规模数据处理效率。通过实际运行 WordCount 程序，我进一步理解了并行计算任务划分、进程通信以及结果汇总等关键环节。

实验过程中也遇到了许多实际问题，例如虚拟机网络配置、主机名与 IP 地址映射、SSH 连接异常以及 MPI 节点通信失败等。通过查阅资料 and 不断调试，我逐步掌握了集群环境故障排查的方法，提高了 Linux 系统操作能力和解决实际问题的能力。本次实验不仅加深了我对高性能计算与分布式系统的理解，也让我认识到环境配置和系统部署在工程实践中的重要性，为今后学习并行程序设计和大数据处理技术奠定了良好的基础。

## 5. 思考题

### MPI 的分布式计算原理：

MPI (Message Passing Interface, 消息传递接口) 是一种用于分布式并行计算的标准编程模型。在 MPI 环境中，每个计算节点运行独立的进程，各进程拥有自己的地址空间，不能直接访问其他进程的数据。当进程之间需要交换信息时，必须通过 MPI 提供的消息传递机制进行通信。

MPI 程序启动后，会生成多个并行进程，每个进程拥有唯一的进程编号 (Rank)。程序运行过程中，任务首先被划分为多个子任务，并分配给不同进程执行。各进程完成自身计算任务后，通过 MPI\_Send、MPI\_Recv、MPI\_Bcast、MPI\_Reduce 等通信函数实现数据交换与结果汇总，最终由主进程收集并输出计算结果。MPI 采用“任务划分—并行计算—消息通信—结果汇总”的工作模式，从而充分利用多台计算机的计算资源，实现大规模数据的并行处理。

### MPI 的特点：

### **(1) 高性能**

MPI 采用进程间消息传递机制，通信开销较小，能够充分利用多核处理器和集群计算资源，适用于高性能计算场景。

### **(2) 良好的可扩展性**

MPI 程序可以运行在单机多进程环境，也可以扩展到由数百甚至数千个节点组成的大规模集群，具有较强的扩展能力。

### **(3) 分布式存储结构**

每个 MPI 进程拥有独立的内存空间，避免了共享内存环境中的资源竞争问题，提高了程序运行的稳定性。

### **(4) 跨平台支持**

MPI 已成为国际通用的并行计算标准，可运行于 Linux、Windows 以及各种高性能计算平台，具有良好的可移植性。

### **(5) 编程灵活**

MPI 提供丰富的通信函数和并行控制机制，支持点对点通信、广播通信、集合通信等多种通信方式，能够满足不同类型并行程序的开发需求。

因此，MPI 广泛应用于科学计算、大数据分析、人工智能训练、气象模拟以及工程仿真等需要大规模并行计算的领域，是当前高性能计算领域最重要的并行编程标准之一。